Практическая работа № 3

Тема: Приёмы надёжного программирования

Тема курсовой работы: Система управления зоомагазином

Карась Полина Алексеевна

Задание 1. Модель данных с валидацией

Для задания выбрана сущность «Товар» из диаграммы классов (практическая работа №1), так как она используется в каталоге, на складе и в продажах. Поля: номер, название, цена, количество и срок годности.

```
public class Product
{
   [Range(1, int.MaxValue, ErrorMessage = "Не указан номер товара")]
   public int Id { get; set; }

   [Required(ErrorMessage = "Не указано название товара")]
   [StringLength(100, MinimumLength = 2, ErrorMessage = "Название должно быть от 2 до 100 символов")]
   public string Name { get; set; } = string.Empty;

   [Range(0.01, 1000000.0, ErrorMessage = "Цена должна быть больше нуля")]
   public decimal Price { get; set; }

   [Range(0, 100000, ErrorMessage = "Количество не может быть отрицательным")]
   public int Quantity { get; set; }

   [Required(ErrorMessage = "Не указан срок годности")]
   public DateTime? ExpirationDate { get; set; }
}
```

```
 Проверка данных товара
Не указан номер товара
Не указано название товара
Цена должна быть больше нуля
Количество не может быть отрицательным
Не указан срок годности
```

Объяснение атрибутов:

Поле	**Атрибут**	**Зачем**
Id	Range(1, int.MaxValue)	Номер товара должен быть больше нуля
Name	Required, StringLength(2–100)	Название нельзя оставить пустым; длина ограничена
Price	Range(0.01 – 1 000 000)	Цена не может быть нулевой или отрицательной
Quantity	Range(0 – 100 000)	Остаток на складе не может быть отрицательным
ExpirationDate	Required	У корма должен быть указан срок годности

Проверка запускается вручную через Validator.TryValidateObject, потому что сами атрибуты ничего не проверяют:

```
var context = new ValidationContext(product);
var results = new List<ValidationResult>();
bool isValid = Validator.TryValidateObject(product, context, results, validateAllProperties: true);
if (!isValid)
{
   foreach (var error in results)
```

```
        Console.WriteLine(error.ErrorMessage);
}
```

Задание 2. Метод с обработкой исключений

Оформить продажу и изменить остаток (ключевой сценарий из практической работы №2). Если не проверить данные, можно продать несуществующий или просроченный товар, либо уменьшить остаток ниже нуля.

Пользовательские исключения:

Исключение	**Когда возникает**
ProductNotFoundException	товар не найден в каталоге
ProductExpiredException	у товара истёк срок годности
NotEnoughStockException	на складе меньше товара, чем нужно продать

```
public class ProductNotFoundException : Exception
{
   public int ProductId { get; }

   public ProductNotFoundException(int productId)
      : base($"Товар {productId} не найден")
   {
      ProductId = productId;
   }
}
```

Остальные два исключения написаны так же (в коде программы).

Метод оформления продажи:

```
public void MakeSale(int productId, int quantity)
{
   try
   {
      if (!_products.ContainsKey(productId))
         throw new ProductNotFoundException(productId);

      Product product = _products[productId];

      if (product.ExpirationDate < DateTime.Today)
         throw new ProductExpiredException(productId);

      if (product.Quantity < quantity)
         throw new NotEnoughStockException(productId, quantity, product.Quantity);

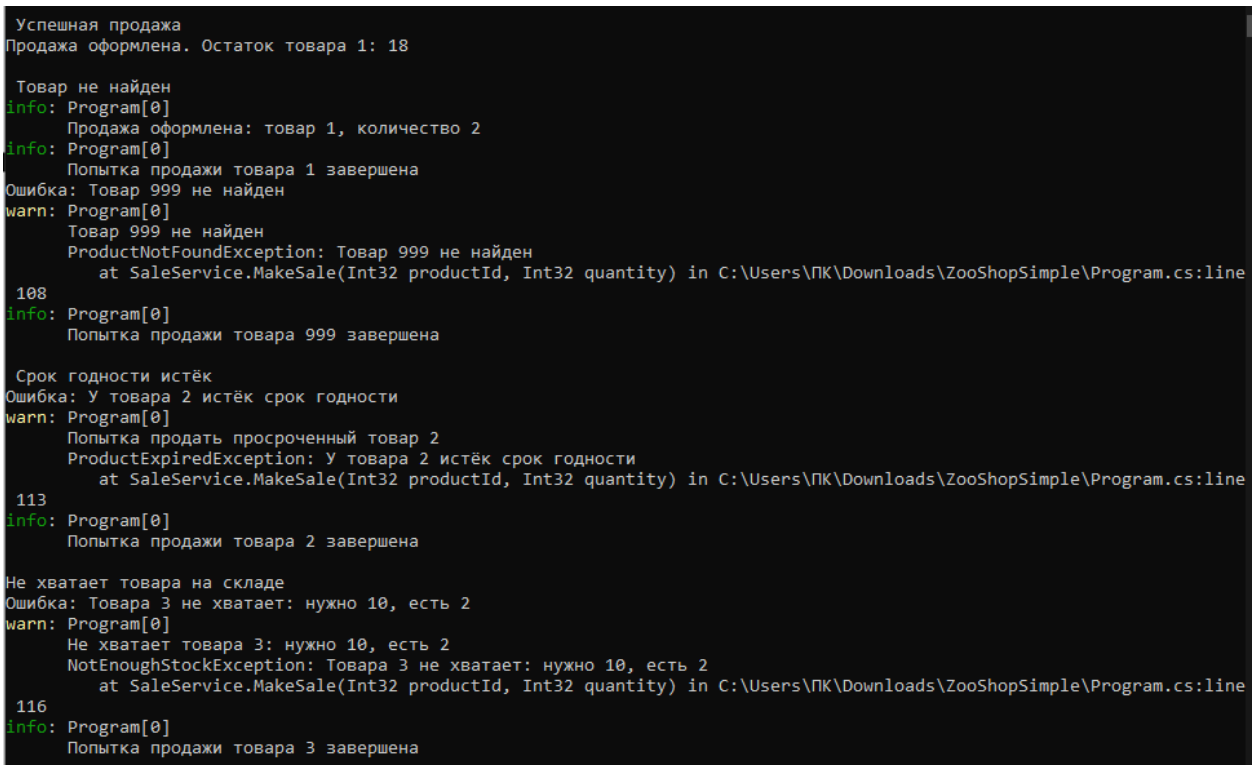
      product.Quantity -= quantity;
      Console.WriteLine($"Продажа оформлена. Остаток товара {productId}: {product.Quantity}");
      _logger.LogInformation("Продажа оформлена: товар {ProductId}, количество {Quantity}",
productId, quantity);
   }
   catch (ProductNotFoundException ex)
   {
      Console.WriteLine("Ошибка: " + ex.Message);
```

```
        _logger.LogWarning(ex, "Товар {ProductId} не найден", ex.ProductId);
    }
    catch (ProductExpiredException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
        _logger.LogWarning(ex, "Попытка продать просроченный товар {ProductId}", ex.ProductId);
    }
    catch (NotEnoughStockException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
        _logger.LogWarning(ex, "Не хватает товара {ProductId}: нужно {Requested}, есть {Available}",
            ex.ProductId, ex.Requested, ex.Available);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Непредвиденная ошибка при продаже товара {ProductId}", productId);
        throw;
    }
    finally
    {
        _logger.LogInformation("Попытка продажи товара {ProductId} завершена", productId);
    }
}
```

Что делает каждый блок:

Блок	**Действие**
catch ProductNotFoundException	сообщает, что товар не найден; пишет предупреждение в лог
catch ProductExpiredException	сообщает о просроченном товаре; пишет предупреждение в лог
catch NotEnoughStockException	сообщает, что не хватает товара; пишет предупреждение в лог
catch Exception	пишет ошибку в лог и передаёт её выше (throw;)
finally	выполняется всегда: пишет в лог, что попытка продажи завершена

Более конкретные catch стоят выше общего catch (Exception). Остаток уменьшается только после всех проверок, поэтому при ошибке он не меняется.



Задание 3. Логирование

В метод MakeSale добавлены вызовы логирования трёх уровней. Данные передаются именованными параметрами в фигурных скобках, чтобы по ним можно было искать записи в логе.

Вызов	**Уровень**	**Именованные параметры**	**Почему этот уровень**
«Продажа оформлена: товар {ProductId}, количество {Quantity}»	Information	ProductId, Quantity	обычное успешное событие
«Попытка продажи товара {ProductId} завершена» (в finally)	Information	ProductId	фиксирует конец попытки при любом исходе
«Товар {ProductId} не найден»	Warning	ProductId	нештатная, но ожидаемая ситуация, программа продолжает работу
«Попытка продать просроченный товар {ProductId}»	Warning	ProductId	сработал запрет продажи, это не сбой системы
«Не хватает товара {ProductId}: нужно {Requested}, есть {Available}»	Warning	ProductId, Requested, Available	ожидаемый отказ; числа показывают, насколько не хватает
«Непредвиденная ошибка при продаже товара {ProductId}»	Error	ProductId (и сама ошибка ex)	операция не выполнена из-за неизвестного сбоя

Задание 4. Меры повышения надёжности

«Продажи и остатки». Выбраны две меры

Мера 1. Проверка данных перед продажей. Перед уменьшением остатка проверяется, что товар есть в каталоге, срок годности не истёк и товара хватает. Что компенсирует: ошибки пользователя и неверные данные -система не позволит продать несуществующий или просроченный товар и увести остаток в минус.

Мера 2. Транзакция с блокировкой остатка. Проверка наличия, сохранение продажи и уменьшение остатка выполняются в одной транзакции базы данных; при ошибке всё откатывается. Что компенсирует: одновременную работу двух продавцов с одним товаром и частичные сбои (остаток уменьшен, а продажа не сохранена). Это решение из практической работы №2.

Задание 5. Фиксация результатов в репозитории

Код помещён в папку src репозитория. Выполнены два коммита:

Ссылки на коммиты:

1. https://github.com/konserrva/zoomagazin-karas/commit/4519e8fb0661f7808919f980b909b3ecf04d69af
2. https://github.com/konserrva/zoomagazin-karas/commit/2fba590d29536dfc7797ab8e3db4e895ead005d1

Вывод

Освоила приёмы надёжного программирования: проверку данных через Data Annotations, обработку исключений с собственными классами ошибок и логирование через ILogger.